

Matrix Multiplication Algorithms: Pseudocode, Author-Provided Source Code, and CPU Implementations

Literature Survey & Companion Code — NIT Warangal Research Internship

August 16, 2026

Abstract

This document consolidates, for each of the 15 matrix multiplication algorithms covered in the literature survey: (1) algorithmic pseudocode reflecting the verified original publication, (2) the actual author-provided original source code where one exists and is publicly available, and (3) a working CPU (C) implementation, correctness-tested against the classical result. Where an algorithm has no genuine author-released code (the large majority — most predate the era of authors publishing runnable source, or are theoretical/umbrella categories with no single implementation), or is inherently distributed-memory / GPU-only and had to be adapted or simulated to run on a single CPU, this is stated explicitly rather than left implicit.

Contents

1	Classical (Naïve) Matrix Multiplication	2
2	Strassen’s Algorithm	3
3	Winograd’s Algorithm (Strassen–Winograd Variant)	6
4	Coppersmith–Winograd Algorithm	8
5	Schönhage–Strassen Algorithm (Integer Multiplication – Included Per Requirement)	9
6	Block (Blocked) Matrix Multiplication	11
7	Tiled Matrix Multiplication (CUDA Shared-Memory Tiling)	12
8	Recursive Matrix Multiplication (Plain Divide-and-Conquer)	14
9	Cache-Oblivious Matrix Multiplication	16
10	Cannon’s Algorithm	18
11	Fox’s Algorithm (Broadcast-Multiply-Roll)	20
12	SUMMA (Scalable Universal Matrix Multiplication Algorithm)	22
13	Communication-Avoiding Matrix Multiplication (CA-MM / 2.5D)	25
14	Parallel GEMM (General Category)	26
15	cuBLAS GEMM (Optimized Library Implementation – Not a Novel Algorithm)	27

1 Classical (Naïve) Matrix Multiplication

Pseudocode

Algorithm 1 Classical Matrix Multiplication

```
1: function CLASSICALMATMUL( $A, B$ )  $\triangleright A$  is  $m \times n$ ,  $B$  is  $n \times p$ 
2:   Initialize  $C$  as an  $m \times p$  matrix of zeros
3:   for  $i \leftarrow 1$  to  $m$  do
4:     for  $j \leftarrow 1$  to  $p$  do
5:        $sum \leftarrow 0$ 
6:       for  $k \leftarrow 1$  to  $n$  do
7:          $sum \leftarrow sum + A[i][k] \cdot B[k][j]$ 
8:       end for
9:        $C[i][j] \leftarrow sum$ 
10:    end for
11:  end for
12:  return  $C$ 
13: end function
```

CPU (C) Implementation — This Project

```
1  1. CLASSICAL (NAIVE) MATRIX MULTIPLICATION -- O(n^3)
2  ===== */
3  void classical_matmul(double** A, double** B, double** C, int n) {
4      for (int i = 0; i < n; i++) {
5          for (int j = 0; j < n; j++) {
6              double sum = 0.0;
7              for (int k = 0; k < n; k++) {
8                  sum += A[i][k] * B[k][j];
9              }
10             C[i][j] = sum;
11         }
12     }
13 }
14
15 /* =====
```

Listing 1: CPU implementation

2 Strassen's Algorithm

Pseudocode

Algorithm 2 Strassen's Recursive Matrix Multiplication

```
1: function STRASSEN( $A, B$ ) ▷  $A, B$  are  $n \times n$ ,  $n$  a power of 2
2:   if  $n = 1$  then
3:     return  $A \times B$  ▷ scalar multiplication
4:   end if
5:   Partition  $A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$ 
6:    $M_1 \leftarrow \text{STRASSEN}(A_{11} + A_{22}, B_{11} + B_{22})$ 
7:    $M_2 \leftarrow \text{STRASSEN}(A_{21} + A_{22}, B_{11})$ 
8:    $M_3 \leftarrow \text{STRASSEN}(A_{11}, B_{12} - B_{22})$ 
9:    $M_4 \leftarrow \text{STRASSEN}(A_{22}, B_{21} - B_{11})$ 
10:   $M_5 \leftarrow \text{STRASSEN}(A_{11} + A_{12}, B_{22})$ 
11:   $M_6 \leftarrow \text{STRASSEN}(A_{21} - A_{11}, B_{11} + B_{12})$ 
12:   $M_7 \leftarrow \text{STRASSEN}(A_{12} - A_{22}, B_{21} + B_{22})$ 
13:   $C_{11} \leftarrow M_1 + M_4 - M_5 + M_7$ 
14:   $C_{12} \leftarrow M_3 + M_5$ 
15:   $C_{21} \leftarrow M_2 + M_4$ 
16:   $C_{22} \leftarrow M_1 - M_2 + M_3 + M_6$ 
17:  return  $C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$ 
18: end function
```

CPU (C) Implementation — This Project

```
1  2. STRASSEN'S ALGORITHM -- O(n*log2(7)) ~ O(n^2.807)
2  Requires n to be a power of 2. Falls back to classical multiplication
3  below a base-case threshold (standard practical optimization, since
4  Strassen's recursive overhead outweighs its FLOP savings for small n).
5  ===== */
6  #define STRASSEN_BASE_CASE 64
7
8  static double** strassen_recursive(double** A, double** B, int n) {
9      double** C = alloc_matrix(n);
10
11     if (n <= STRASSEN_BASE_CASE) {
12         classical_matmul(A, B, C, n);
13         return C;
14     }
15
16     int half = n / 2;
17
18     /* Allocate quadrant sub-matrices */
19     double **A11 = alloc_matrix(half), **A12 = alloc_matrix(half);
20     double **A21 = alloc_matrix(half), **A22 = alloc_matrix(half);
21     double **B11 = alloc_matrix(half), **B12 = alloc_matrix(half);
22     double **B21 = alloc_matrix(half), **B22 = alloc_matrix(half);
23
24     for (int i = 0; i < half; i++) {
25         for (int j = 0; j < half; j++) {
26             A11[i][j] = A[i][j];
27             A12[i][j] = A[i][j + half];
28             A21[i][j] = A[i + half][j];
29             A22[i][j] = A[i + half][j + half];
30             B11[i][j] = B[i][j];
31             B12[i][j] = B[i][j + half];
32             B21[i][j] = B[i + half][j];
33             B22[i][j] = B[i + half][j + half];
34         }
35     }
```

```

35 }
36
37 double** S1 = matrix_add(A11, A22, half);
38 double** T1 = matrix_add(B11, B22, half);
39 double** M1 = strassen_recursive(S1, T1, half);
40
41 double** S2 = matrix_add(A21, A22, half);
42 double** M2 = strassen_recursive(S2, B11, half);
43
44 double** T3 = matrix_sub(B12, B22, half);
45 double** M3 = strassen_recursive(A11, T3, half);
46
47 double** T4 = matrix_sub(B21, B11, half);
48 double** M4 = strassen_recursive(A22, T4, half);
49
50 double** S5 = matrix_add(A11, A12, half);
51 double** M5 = strassen_recursive(S5, B22, half);
52
53 double** S6 = matrix_sub(A21, A11, half);
54 double** T6 = matrix_add(B11, B12, half);
55 double** M6 = strassen_recursive(S6, T6, half);
56
57 double** S7 = matrix_sub(A12, A22, half);
58 double** T7 = matrix_add(B21, B22, half);
59 double** M7 = strassen_recursive(S7, T7, half);
60
61 /* C11 = M1 + M4 - M5 + M7 ; C12 = M3 + M5
62    C21 = M2 + M4 ; C22 = M1 - M2 + M3 + M6 */
63 for (int i = 0; i < half; i++) {
64     for (int j = 0; j < half; j++) {
65         C[i][j] = M1[i][j] + M4[i][j] - M5[i][j] + M7[i][j];
66         C[i][j + half] = M3[i][j] + M5[i][j];
67         C[i + half][j] = M2[i][j] + M4[i][j];
68         C[i + half][j + half] = M1[i][j] - M2[i][j] + M3[i][j] + M6[i][j];
69     }
70 }
71
72 /* Cleanup */
73 double** temps[] = {S1, T1, M1, S2, M2, T3, M3, T4, M4, S5, M5, S6, T6, M6, S7, T7, M7};
74 for (int t = 0; t < 17; t++) free_matrix(temps[t], half);
75 free_matrix(A11, half); free_matrix(A12, half); free_matrix(A21, half); free_matrix(A22, half);
76 free_matrix(B11, half); free_matrix(B12, half); free_matrix(B21, half); free_matrix(B22, half);
77
78 return C;
79 }
80
81 static int next_pow2(int n) {
82     int p = 1;
83     while (p < n) p <= 1;
84     return p;
85 }
86
87 double** strassen_matmul(double** A, double** B, int n) {
88     int padded = next_pow2(n);
89     if (padded == n) {
90         return strassen_recursive(A, B, n);
91     }
92     /* Pad to next power of two with zeros, run, then the caller should
93        only read the top-left n x n block of the result. */
94     double** Apad = alloc_matrix(padded);
95     double** Bpad = alloc_matrix(padded);
96     for (int i = 0; i < n; i++)
97         for (int j = 0; j < n; j++) { Apad[i][j] = A[i][j]; Bpad[i][j] = B[i][j]; }
98     double** Cpad = strassen_recursive(Apad, Bpad, padded);
99     double** C = alloc_matrix(n);
100    for (int i = 0; i < n; i++)
101        for (int j = 0; j < n; j++) C[i][j] = Cpad[i][j];
102    free_matrix(Apad, padded); free_matrix(Bpad, padded); free_matrix(Cpad, padded);
103    return C;
104 }

```

```
105 / * =====  
106
```

Listing 2: CPU implementation

3 Winograd's Algorithm (Strassen-Winograd Variant)

Pseudocode

Algorithm 3 Winograd's 2×2 Block Multiplication (reduced-addition form)

```
1: function WINOGRADBLOCK2X2( $A, B$ )
2:    $S_1 \leftarrow A_{21} + A_{22}$ ;  $S_2 \leftarrow S_1 - A_{11}$ ;  $S_3 \leftarrow A_{11} - A_{21}$ ;  $S_4 \leftarrow A_{12} - S_2$ 
3:    $T_1 \leftarrow B_{12} - B_{11}$ ;  $T_2 \leftarrow B_{22} - T_1$ ;  $T_3 \leftarrow B_{22} - B_{12}$ ;  $T_4 \leftarrow T_2 - B_{21}$ 
4:    $M_1 \leftarrow S_2 \cdot T_2$ ;  $M_2 \leftarrow A_{11} \cdot B_{11}$ ;  $M_3 \leftarrow A_{12} \cdot B_{21}$ 
5:    $M_4 \leftarrow S_3 \cdot T_3$ ;  $M_5 \leftarrow S_1 \cdot T_1$ ;  $M_6 \leftarrow S_4 \cdot B_{22}$ ;  $M_7 \leftarrow A_{22} \cdot T_4$ 
6:    $U_1 \leftarrow M_2 + M_3$ ;  $U_2 \leftarrow M_2 + M_5$ ;  $U_3 \leftarrow U_2 + M_4$ 
7:    $C_{11} \leftarrow U_1$ ;  $C_{12} \leftarrow U_3 + M_6$ ;  $C_{21} \leftarrow U_3 + M_7$ ;  $C_{22} \leftarrow U_2 + M_1$ 
8:   return  $C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$  ▷ Same 7 multiplications as Strassen, but only 15 add/sub instead of 18
9: end function
```

CPU (C) Implementation — This Project

```
1  3. WINOGRAD'S ALGORITHM (Strassen-Winograd variant)
2  Same 7 multiplications as Strassen, but only 15 add/sub instead of 18
3  (S1..S4, T1..T4, then combine -- matches the paper's reduced-addition form).
4  ===== */
5  static double** winograd_recursive(double** A, double** B, int n) {
6      double** C = alloc_matrix(n);
7      if (n <= STRASSEN_BASE_CASE) {
8          classical_matmul(A, B, C, n);
9          return C;
10     }
11     int half = n / 2;
12     double **A11 = alloc_matrix(half), **A12 = alloc_matrix(half);
13     double **A21 = alloc_matrix(half), **A22 = alloc_matrix(half);
14     double **B11 = alloc_matrix(half), **B12 = alloc_matrix(half);
15     double **B21 = alloc_matrix(half), **B22 = alloc_matrix(half);
16     for (int i = 0; i < half; i++)
17         for (int j = 0; j < half; j++) {
18             A11[i][j] = A[i][j];           A12[i][j] = A[i][j + half];
19             A21[i][j] = A[i + half][j];     A22[i][j] = A[i + half][j + half];
20             B11[i][j] = B[i][j];           B12[i][j] = B[i][j + half];
21             B21[i][j] = B[i + half][j];     B22[i][j] = B[i + half][j + half];
22         }
23
24     /* S1 = A21+A22 ; S2 = S1-A11 ; S3 = A11-A21 ; S4 = A12-S2 */
25     double** S1 = matrix_add(A21, A22, half);
26     double** S2 = matrix_sub(S1, A11, half);
27     double** S3 = matrix_sub(A11, A21, half);
28     double** S4 = matrix_sub(A12, S2, half);
29     /* T1 = B12-B11 ; T2 = B22-T1 ; T3 = B22-B12 ; T4 = T2-B21 */
30     double** T1 = matrix_sub(B12, B11, half);
31     double** T2 = matrix_sub(B22, T1, half);
32     double** T3 = matrix_sub(B22, B12, half);
33     double** T4 = matrix_sub(T2, B21, half);
34
35     double** M1 = winograd_recursive(S2, T2, half);
36     double** M2 = winograd_recursive(A11, B11, half);
37     double** M3 = winograd_recursive(A12, B21, half);
38     double** M4 = winograd_recursive(S3, T3, half);
39     double** M5 = winograd_recursive(S1, T1, half);
40     double** M6 = winograd_recursive(S4, B22, half);
41     double** M7 = winograd_recursive(A22, T4, half);
42
43     /* U1 = M2+M3 ; U2 = M2+M5 ; U3 = U2+M4
44        C11 = U1 ; C12 = U3+M6 ; C21 = U3+M7 ; C22 = U2+M1 */
45     for (int i = 0; i < half; i++) {
46         for (int j = 0; j < half; j++) {
47             double U1 = M2[i][j] + M3[i][j];
```

```

48     double U2 = M2[i][j] + M5[i][j];
49     double U3 = U2 + M4[i][j];
50     C[i][j] = U1;
51     C[i][j + half] = U3 + M6[i][j];
52     C[i + half][j] = U3 + M7[i][j];
53     C[i + half][j + half] = U2 + M1[i][j];
54 }
55 }
56
57 double** temps[] = {S1,S2,S3,S4,T1,T2,T3,T4,M1,M2,M3,M4,M5,M6,M7};
58 for (int t = 0; t < 15; t++) free_matrix(temps[t], half);
59 free_matrix(A11, half); free_matrix(A12, half); free_matrix(A21, half); free_matrix(A22, half);
60 free_matrix(B11, half); free_matrix(B12, half); free_matrix(B21, half); free_matrix(B22, half);
61 return C;
62 }
63
64 double** winograd_matmul(double** A, double** B, int n) {
65     int padded = next_pow2(n);
66     if (padded == n) return winograd_recursive(A, B, n);
67     double** Apad = alloc_matrix(padded);
68     double** Bpad = alloc_matrix(padded);
69     for (int i = 0; i < n; i++)
70         for (int j = 0; j < n; j++) { Apad[i][j] = A[i][j]; Bpad[i][j] = B[i][j]; }
71     double** Cpad = winograd_recursive(Apad, Bpad, padded);
72     double** C = alloc_matrix(n);
73     for (int i = 0; i < n; i++)
74         for (int j = 0; j < n; j++) C[i][j] = Cpad[i][j];
75     free_matrix(Apad, padded); free_matrix(Bpad, padded); free_matrix(Cpad, padded);
76     return C;
77 }
78
79 /* =====

```

Listing 3: CPU implementation

4 Coppersmith–Winograd Algorithm

Pseudocode

Algorithm 4 Coppersmith–Winograd (high-level structure; theoretical / galactic algorithm)

- 1: **function** COPPERSMITHWINOGRAD(A, B)
 - 2: Construct a trilinear algebraic identity based on a Salem–Spencer arithmetic-progression-free set of integers
 - 3: Represent the matrix product implicitly as a tensor T with low "border rank" relative to naive multiplication
 - 4: Amplify the base identity via repeated tensor (Kronecker) powers $T^{\otimes k}$ to drive the effective exponent $\omega \rightarrow 2.376$
 - 5: Recover $C = A \times B$ from the amplified tensor decomposition
 - 6: **return** C ▷ Not implemented in practice: hidden constants are astronomically large
 - 7: **end function**
-

Implementation note. No practical implementation of Coppersmith–Winograd exists anywhere (it is a “galactic algorithm”); the CPU implementation below documents this and safely falls back to Strassen’s algorithm rather than fabricating a fake implementation.

CPU (C) Implementation — This Project

```
1  4. COPPERSMITH-WINOGRAD -- THEORETICAL ONLY
2  No practical implementation exists anywhere (it is a "galactic
3  algorithm": the hidden constant factors are so large that it never
4  outperforms even classical multiplication for any matrix size that
5  could ever be materialized). Per the survey's own findings, we do
6  NOT fabricate a fake "CW implementation" -- that would misrepresent
7  the algorithm. Instead this function documents that fact and safely
8  falls back to Strassen (the best PRACTICAL sub-cubic algorithm)
9  so the driver program can still run end-to-end.
10 ===== */
11 double** coppersmith_winograd_matmul(double** A, double** B, int n) {
12     fprintf(stderr,
13         "[NOTE] Coppersmith-Winograd (1990, exponent 2.376) has no practical "
14         "implementation on any hardware -- its hidden constants are "
15         "astronomically large ('galactic algorithm'). Falling back to "
16         "Strassen's algorithm as the best PRACTICAL sub-cubic substitute.\n");
17     return strassen_matmul(A, B, n);
18 }
19
20 /* =====
```

Listing 4: CPU implementation

5 Schönhage–Strassen Algorithm (Integer Multiplication – Included Per Requirement)

Pseudocode

Algorithm 5 Schönhage–Strassen Fast Integer Multiplication (NOT matrix multiplication)

```
1: function SCHONHAGESTRASSEN( $x, y$ ) ▷  $x, y$  are  $N$ -digit integers
2:   Split  $x, y$  into digit blocks; treat as polynomial coefficient vectors
3:    $\hat{X} \leftarrow \text{FFT}(x), \hat{Y} \leftarrow \text{FFT}(y)$  ▷ modular FFT over  $\mathbb{Z}/(2^N + 1)\mathbb{Z}$ 
4:    $\hat{Z} \leftarrow \hat{X} \cdot \hat{Y}$  ▷ pointwise multiplication (convolution theorem)
5:    $z \leftarrow \text{InverseFFT}(\hat{Z})$ 
6:   Perform carry propagation on  $z$  to obtain the final integer product
7:   return  $z$ 
8: end function
```

Implementation note. Schönhage–Strassen is an *integer*-multiplication algorithm, not a matrix-multiplication algorithm. The CPU implementation below is therefore written on its actual domain (big-integer multiplication via FFT-based convolution), rather than being forced into a matrix-multiplication interface it was never designed for.

CPU (C) Implementation — This Project

```
1  5. SCHONHAGE-STRASSEN -- INTEGER multiplication, NOT matrix multiplication
2  Multiplies two big integers (given as arrays of decimal digits, most
3  significant digit first) via a simplified FFT-based convolution,
4  demonstrating the algorithm on the domain it actually applies to.
5  This is a compact, correctness-focused DFT implementation (O(n^2)
6  naive DFT, not the full O(n log n) radix-2 FFT) purely for
7  pedagogical/demonstration purposes within this CPU reference project.
8  ===== */
9  typedef struct { double re, im; } Complex;
10
11 static Complex c_add(Complex a, Complex b) { Complex r = {a.re+b.re, a.im+b.im}; return r; }
12 static Complex c_mul(Complex a, Complex b) {
13     Complex r = { a.re*b.re - a.im*b.im, a.re*b.im + a.im*b.re };
14     return r;
15 }
16
17 /* Naive O(n^2) DFT / inverse-DFT (sufficient for demonstrating the
18    convolution-based multiplication principle at small digit counts) */
19 static void dft(Complex* in, Complex* out, int n, int invert) {
20     for (int k = 0; k < n; k++) {
21         Complex sum = {0.0, 0.0};
22         for (int t = 0; t < n; t++) {
23             double angle = (invert ? 2.0 : -2.0) * M_PI * k * t / n;
24             Complex w = { cos(angle), sin(angle) };
25             sum = c_add(sum, c_mul(in[t], w));
26         }
27         if (invert) { sum.re /= n; sum.im /= n; }
28         out[k] = sum;
29     }
30 }
31
32 void schonhage_strassen_multiply(int* x, int lenx, int* y, int leny,
33                                 int* result, int* result_len) {
34     int conv_len = lenx + leny - 1;
35     int fft_len = 1;
36     while (fft_len < conv_len) fft_len <<= 1;
37     fft_len <<= 1; /* extra padding to reduce aliasing in the naive DFT */
38
39     Complex* X = (Complex*)calloc(fft_len, sizeof(Complex));
40     Complex* Y = (Complex*)calloc(fft_len, sizeof(Complex));
41     Complex* Xf = (Complex*)calloc(fft_len, sizeof(Complex));
42     Complex* Yf = (Complex*)calloc(fft_len, sizeof(Complex));
```

```

43 Complex* Zf = (Complex*)calloc(fft_len, sizeof(Complex));
44 Complex* Z = (Complex*)calloc(fft_len, sizeof(Complex));
45
46 /* least-significant digit first for convolution convenience */
47 for (int i = 0; i < lenx; i++) X[i].re = x[lenx - 1 - i];
48 for (int i = 0; i < leny; i++) Y[i].re = y[leny - 1 - i];
49
50 dft(X, Xf, fft_len, 0);
51 dft(Y, Yf, fft_len, 0);
52 for (int i = 0; i < fft_len; i++) Zf[i] = c_mul(Xf[i], Yf[i]);
53 dft(Zf, Z, fft_len, 1);
54
55 /* Round to nearest integer per convolution coefficient, then
56 propagate carries to obtain the final base-10 product */
57 int* digits = (int*)calloc(fft_len + 2, sizeof(int));
58 long long carry = 0;
59 for (int i = 0; i < fft_len; i++) {
60     long long val = (long long)llround(Z[i].re) + carry;
61     digits[i] = (int)(val % 10);
62     carry = val / 10;
63     if (digits[i] < 0) { digits[i] += 10; carry -= 1; }
64 }
65 int pos = fft_len;
66 while (carry > 0) { digits[pos++] = (int)(carry % 10); carry /= 10; }
67
68 /* strip leading zeros (from the most-significant end) */
69 int top = pos - 1;
70 while (top > 0 && digits[top] == 0) top--;
71
72 *result_len = top + 1;
73 for (int i = 0; i < *result_len; i++) result[i] = digits[top - i];
74
75 free(X); free(Y); free(Xf); free(Yf); free(Zf); free(Z); free(digits);
76 }
77
78 /* =====

```

Listing 5: CPU implementation

6 Block (Blocked) Matrix Multiplication

Pseudocode

Algorithm 6 Blocked Matrix Multiplication (cache-tiled)

```
1: function BLOCKMATMUL( $A, B, b$ ) ▷  $b$  = block size fitting in cache
2:   Initialize  $C$  as an  $n \times n$  matrix of zeros
3:   for  $ii \leftarrow 1$  to  $n$  step  $b$  do
4:     for  $jj \leftarrow 1$  to  $n$  step  $b$  do
5:       for  $kk \leftarrow 1$  to  $n$  step  $b$  do
6:         for  $i \leftarrow ii$  to  $\min(ii + b - 1, n)$  do
7:           for  $j \leftarrow jj$  to  $\min(jj + b - 1, n)$  do
8:              $sum \leftarrow 0$ 
9:             for  $k \leftarrow kk$  to  $\min(kk + b - 1, n)$  do
10:               $sum \leftarrow sum + A[i][k] \cdot B[k][j]$ 
11:            end for
12:             $C[i][j] \leftarrow C[i][j] + sum$ 
13:          end for
14:        end for
15:      end for
16:    end for
17:  end for
18:  return  $C$ 
19: end function
```

CPU (C) Implementation — This Project

```
1  6. BLOCK (BLOCKED) MATRIX MULTIPLICATION -- cache-tiled, O(n^3)
2  ===== */
3  void block_matmul(double** A, double** B, double** C, int n, int block_size) {
4    fill_zero(C, n);
5    for (int ii = 0; ii < n; ii += block_size) {
6      for (int jj = 0; jj < n; jj += block_size) {
7        for (int kk = 0; kk < n; kk += block_size) {
8          int i_max = (ii + block_size < n) ? ii + block_size : n;
9          int j_max = (jj + block_size < n) ? jj + block_size : n;
10         int k_max = (kk + block_size < n) ? kk + block_size : n;
11         for (int i = ii; i < i_max; i++) {
12           for (int j = jj; j < j_max; j++) {
13             double sum = 0.0;
14             for (int k = kk; k < k_max; k++) {
15               sum += A[i][k] * B[k][j];
16             }
17             C[i][j] += sum;
18           }
19         }
20       }
21     }
22   }
23 }
```

Listing 6: CPU implementation

7 Tiled Matrix Multiplication (CUDA Shared-Memory Tiling)

Pseudocode

Algorithm 7 CUDA Tiled GEMM Kernel (per thread block)

```
1: function TILEDMATMULKERNEL( $A, B, C, n, T$ ) ▷  $T$  = tile width; runs per CUDA thread
2:    $row \leftarrow blockIdx.y \cdot T + threadIdx.y$ 
3:    $col \leftarrow blockIdx.x \cdot T + threadIdx.x$ 
4:    $\_\_shared\_\_ As[T][T], Bs[T][T]$ 
5:    $sum \leftarrow 0$ 
6:   for  $ph \leftarrow 0$  to  $(n/T) - 1$  do ▷ loop over tile "phases"
7:      $As[threadIdx.y][threadIdx.x] \leftarrow A[row][ph \cdot T + threadIdx.x]$ 
8:      $Bs[threadIdx.y][threadIdx.x] \leftarrow B[ph \cdot T + threadIdx.y][col]$ 
9:      $\_\_syncthreads()$ 
10:    for  $k \leftarrow 0$  to  $T - 1$  do
11:       $sum \leftarrow sum + As[threadIdx.y][k] \cdot Bs[k][threadIdx.x]$ 
12:    end for
13:     $\_\_syncthreads()$ 
14:  end for
15:   $C[row][col] \leftarrow sum$ 
16: end function
```

Implementation note. Tiled matrix multiplication is a CUDA shared-memory pattern. Since this deliverable targets CPU only, the implementation below *simulates* the exact phase-by-phase tile-load / synchronize / compute structure of a real CUDA kernel using CPU arrays in place of `__shared__` memory.

CPU (C) Implementation — This Project

```
1  7. TILED MATRIX MULTIPLICATION -- CUDA shared-memory tiling pattern,
2  SIMULATED ON CPU.
3
4  On a real GPU, each CUDA thread block cooperatively loads a TxT tile
5  of A and a TxT tile of B into __shared__ memory, synchronizes with
6  __syncthreads(), then every thread in the block reuses those tiles
7  for T multiply-adds before the next tile "phase" is loaded. Here we
8  reproduce that exact phase structure on the CPU using explicit tile
9  buffers (playing the role of shared memory) so the algorithm's data
10 movement pattern matches the CUDA kernel exactly -- only the
11 execution model (sequential CPU loop vs. thousands of parallel GPU
12 threads) differs. The real .cu kernel is given separately.
13 ===== */
14 void tiled_matmul_cpu_simulated(double** A, double** B, double** C, int n, int tile_size) {
15     fill_zero(C, n);
16     double** As = alloc_matrix(tile_size); /* plays the role of __shared__ As */
17     double** Bs = alloc_matrix(tile_size); /* plays the role of __shared__ Bs */
18
19     for (int row_tile = 0; row_tile < n; row_tile += tile_size) {
20         for (int col_tile = 0; col_tile < n; col_tile += tile_size) {
21             int rmax = (row_tile + tile_size < n) ? row_tile + tile_size : n;
22             int cmax = (col_tile + tile_size < n) ? col_tile + tile_size : n;
23
24             /* accumulator per output element in this output tile */
25             int rsize = rmax - row_tile, csize = cmax - col_tile;
26             double** acc = alloc_matrix(tile_size);
27
28             int num_phases = (n + tile_size - 1) / tile_size;
29             for (int ph = 0; ph < num_phases; ph++) {
30                 int k0 = ph * tile_size;
31                 int kmax = (k0 + tile_size < n) ? k0 + tile_size : n;
32                 int ksize = kmax - k0;
```

```

34      /* "Load" phase: cooperative tile load into shared memory */
35      for (int i = 0; i < rsize; i++)
36          for (int k = 0; k < ksize; k++)
37              As[i][k] = A[row_tile + i][k0 + k];
38      for (int k = 0; k < ksize; k++)
39          for (int j = 0; j < csize; j++)
40              Bs[k][j] = B[k0 + k][col_tile + j];
41      /* __syncthreads() equivalent: tiles are now fully loaded */
42
43      /* "Compute" phase: every thread reuses the shared tiles */
44      for (int i = 0; i < rsize; i++)
45          for (int j = 0; j < csize; j++) {
46              double sum = 0.0;
47              for (int k = 0; k < ksize; k++)
48                  sum += As[i][k] * Bs[k][j];
49              acc[i][j] += sum;
50          }
51      /* second __syncthreads() equivalent before next phase */
52  }
53
54      for (int i = 0; i < rsize; i++)
55          for (int j = 0; j < csize; j++)
56              C[row_tile + i][col_tile + j] = acc[i][j];
57
58      free_matrix(acc, tile_size);
59  }
60  }
61  free_matrix(As, tile_size);
62  free_matrix(Bs, tile_size);
63  }
64
65  /* =====

```

Listing 7: CPU implementation

8 Recursive Matrix Multiplication (Plain Divide-and-Conquer)

Pseudocode

Algorithm 8 Plain Recursive Matrix Multiplication

```
1: function RECURSIVEMATMUL( $A, B$ )  $\triangleright n \times n, n$  a power of 2
2:   if  $n = 1$  then
3:     return  $A \times B$ 
4:   end if
5:   Partition  $A, B$  into four  $(n/2) \times (n/2)$  quadrants each
6:    $C_{11} \leftarrow \text{RECURSIVEMATMUL}(A_{11}, B_{11}) + \text{RECURSIVEMATMUL}(A_{12}, B_{21})$ 
7:    $C_{12} \leftarrow \text{RECURSIVEMATMUL}(A_{11}, B_{12}) + \text{RECURSIVEMATMUL}(A_{12}, B_{22})$ 
8:    $C_{21} \leftarrow \text{RECURSIVEMATMUL}(A_{21}, B_{11}) + \text{RECURSIVEMATMUL}(A_{22}, B_{21})$ 
9:    $C_{22} \leftarrow \text{RECURSIVEMATMUL}(A_{21}, B_{12}) + \text{RECURSIVEMATMUL}(A_{22}, B_{22})$ 
10:  return  $C$  assembled from quadrants
11: end function
```

CPU (C) Implementation — This Project

```
1  8. RECURSIVE MATRIX MULTIPLICATION -- plain divide-and-conquer,
2  NO Strassen trick (8 recursive multiplications, not 7),  $O(n^3)$ .
3  Requires  $n$  to be a power of 2 for clean quadrant splitting; the
4  wrapper below pads non-power-of-2 sizes with zeros.
5  ===== */
6  #define RECURSIVE_BASE_CASE 64
7
8  static void recursive_matmul_inplace(double** A, double** B, double** C,
9                                     int ra, int ca, int rb, int cb, int rc, int cc,
10                                    int n, int lda, int ldb, int ldc) {
11
12      if (n <= RECURSIVE_BASE_CASE) {
13          for (int i = 0; i < n; i++) {
14              for (int j = 0; j < n; j++) {
15                  double sum = C[rc+i][cc+j];
16                  for (int k = 0; k < n; k++)
17                      sum += A[ra+i][ca+k] * B[rb+k][cb+j];
18                  C[rc+i][cc+j] = sum;
19              }
20          }
21          return;
22      }
23      int h = n / 2;
24      /* C11 += A11*B11 + A12*B21 ; C12 += A11*B12 + A12*B22
25       C21 += A21*B11 + A22*B21 ; C22 += A21*B12 + A22*B22 */
26      recursive_matmul_inplace(A, B, C, ra, ca, rb, cb, rc, cc, h, lda, ldb, ldc);
27      recursive_matmul_inplace(A, B, C, ra, ca+h, rb+h, cb, rc, cc, h, lda, ldb, ldc);
28      recursive_matmul_inplace(A, B, C, ra, ca, rb, cb+h, rc, cc+h, h, lda, ldb, ldc);
29      recursive_matmul_inplace(A, B, C, ra, ca+h, rb+h, cb+h, rc, cc+h, h, lda, ldb, ldc);
30      recursive_matmul_inplace(A, B, C, ra+h, ca+h, rb+h, cb, rc+h, cc, h, lda, ldb, ldc);
31      recursive_matmul_inplace(A, B, C, ra+h, ca, rb, cb+h, rc+h, cc+h, h, lda, ldb, ldc);
32      recursive_matmul_inplace(A, B, C, ra+h, ca+h, rb+h, cb+h, rc+h, cc+h, h, lda, ldb, ldc);
33  }
34
35  static int next_pow2_local(int n) { int p = 1; while (p < n) p <= 1; return p; }
36
37  void recursive_matmul(double** A, double** B, double** C, int n) {
38      int padded = next_pow2_local(n);
39      if (padded == n) {
40          fill_zero(C, n);
41          recursive_matmul_inplace(A, B, C, 0,0,0,0,0,0, n, n, n, n);
42          return;
43      }
44      double** Apad = alloc_matrix(padded);
45      double** Bpad = alloc_matrix(padded);
```

```

46 double** Cpad = alloc_matrix(padded);
47 for (int i = 0; i < n; i++)
48     for (int j = 0; j < n; j++) { Apad[i][j] = A[i][j]; Bpad[i][j] = B[i][j]; }
49 recursive_matmul_inplace(Apad, Bpad, Cpad, 0,0,0,0,0, padded, padded, padded, padded);
50 for (int i = 0; i < n; i++)
51     for (int j = 0; j < n; j++) C[i][j] = Cpad[i][j];
52 free_matrix(Apad, padded); free_matrix(Bpad, padded); free_matrix(Cpad, padded);
53 }
54
55 /* =====

```

Listing 8: CPU implementation

9 Cache-Oblivious Matrix Multiplication

Pseudocode

Algorithm 9 Cache-Oblivious Matrix Multiplication (Frigo, Leiserson, Prokop, Ramachandran, 1999)

```
1: function CACHEOBLIVIOUSMATMUL( $A, B, m, n, p$ ) ▷  $A$  is  $m \times n$ ,  $B$  is  $n \times p$ 
2:   if  $\max(m, n, p) \leq$  base-case threshold then
3:     Compute  $C \leftarrow A \times B$  directly (classical triple loop)
4:     return  $C$ 
5:   end if
6:   if  $m$  is the largest dimension then
7:     Split  $A$  horizontally into  $A_1, A_2$ ; recurse on each half, concatenate results
8:   else if  $p$  is the largest dimension then
9:     Split  $B$  vertically into  $B_1, B_2$ ; recurse on each half, concatenate results
10:  else ▷  $n$  is the largest dimension
11:    Split  $A$  vertically and  $B$  horizontally into halves along  $n$ 
12:     $C \leftarrow \text{CACHEOBLIVIOUSMATMUL}(A_1, B_1, \dots) + \text{CACHEOBLIVIOUSMATMUL}(A_2, B_2, \dots)$ 
13:  end if
14:  return  $C$ 
15: end function
```

CPU (C) Implementation — This Project

```
1  9. CACHE-OBLIVIOUS MATRIX MULTIPLICATION
2  Recursively splits whichever of {m, n, p} is currently largest,
3  with NO tuning parameter -- this is what makes it "cache-oblivious"
4  (Frigo, Leiserson, Prokop, Ramachandran, FOCS 1999). Operates via
5  offsets into full-size backing arrays so no sub-matrix copies are
6  needed (unlike the simpler Strassen/recursive implementations above).
7  ===== */
8  #define CACHE_OBLIVIOUS_BASE 32
9
10 void cache_oblivious_matmul(double** A, double** B, double** C,
11                             int row_off_a, int col_off_a,
12                             int row_off_b, int col_off_b,
13                             int row_off_c, int col_off_c,
14                             int m, int n, int p, int lda, int ldb, int ldc) {
15    (void)lda; (void)ldb; (void)ldc; /* offsets used directly; kept for API clarity */
16
17    if (m <= CACHE_OBLIVIOUS_BASE && n <= CACHE_OBLIVIOUS_BASE && p <= CACHE_OBLIVIOUS_BASE) {
18        for (int i = 0; i < m; i++) {
19            for (int j = 0; j < p; j++) {
20                double sum = C[row_off_c+i][col_off_c+j];
21                for (int k = 0; k < n; k++)
22                    sum += A[row_off_a+i][col_off_a+k] * B[row_off_b+k][col_off_b+j];
23                C[row_off_c+i][col_off_c+j] = sum;
24            }
25        }
26        return;
27    }
28
29    if (m >= n && m >= p) {
30        /* split A horizontally (split m) */
31        int h = m / 2;
32        cache_oblivious_matmul(A, B, C, row_off_a, col_off_a, row_off_b, col_off_b,
33                                row_off_c, col_off_c, h, n, p, 0,0,0);
34        cache_oblivious_matmul(A, B, C, row_off_a+h, col_off_a, row_off_b, col_off_b,
35                                row_off_c+h, col_off_c, m-h, n, p, 0,0,0);
36    } else if (p >= m && p >= n) {
37        /* split B vertically (split p) */
38        int h = p / 2;
39        cache_oblivious_matmul(A, B, C, row_off_a, col_off_a, row_off_b, col_off_b,
40                                row_off_c, col_off_c, m, n, h, 0,0,0);
41    }
```



```

41     cache_oblivious_matmul(A, B, C, row_off_a, col_off_a, row_off_b, col_off_b+h,
42                           row_off_c, col_off_c+h, m, n, p-h, 0,0,0);
43 } else {
44     /* n is largest: split the shared dimension and accumulate both halves */
45     int h = n / 2;
46     cache_oblivious_matmul(A, B, C, row_off_a, col_off_a, row_off_b, col_off_b,
47                           row_off_c, col_off_c, m, h, p, 0,0,0);
48     cache_oblivious_matmul(A, B, C, row_off_a, col_off_a+h, row_off_b+h, col_off_b,
49                           row_off_c, col_off_c, m, n-h, p, 0,0,0);
50 }
51 }
52
53 void cache_oblivious_matmul_wrapper(double** A, double** B, double** C, int n) {
54     fill_zero(C, n);
55     cache_oblivious_matmul(A, B, C, 0,0,0,0,0,0, n, n, n, n, n, n);
56 }

```

Listing 9: CPU implementation

10 Cannon's Algorithm

Pseudocode

Algorithm 10 Cannon's Algorithm (Distributed-Memory, $\sqrt{p} \times \sqrt{p}$ mesh)

```
1: function CANNONMATMUL( $A, B$ ) ▷ executed identically on each processor  $P_{i,j}$ 
2:   Initially skew  $A$ : processor  $P_{i,j}$  holds  $A$ -block shifted left by  $i$ 
3:   Initially skew  $B$ : processor  $P_{i,j}$  holds  $B$ -block shifted up by  $j$ 
4:    $C_{i,j} \leftarrow 0$ 
5:   for  $step \leftarrow 1$  to  $\sqrt{p}$  do
6:      $C_{i,j} \leftarrow C_{i,j} + A_{local} \times B_{local}$ 
7:     Circularly shift  $A_{local}$  one step left (within processor row)
8:     Circularly shift  $B_{local}$  one step up (within processor column)
9:   end for
10:  return  $C_{i,j}$  ▷ full  $C$  is the union of all local blocks
11: end function
```

Implementation note. Cannon's Algorithm is inherently distributed-memory. The CPU implementation below simulates a $q \times q$ virtual processor grid, executing the algorithm's skew-then-shift communication pattern sequentially on a single machine.

CPU (C) Implementation — This Project

```
1  10. CANNON'S ALGORITHM (simulated q x q grid, n must be divisible by q)
2  ===== */
3  void cannon_matmul_simulated(double** A, double** B, double** C, int n, int q) {
4      if (n % q != 0) { fprintf(stderr, "[Cannon] n must be divisible by q\n"); return; }
5      int s = n / q;
6      fill_zero(C, n);
7
8      /* Extract original blocks */
9      double*** Ablk = (double***)malloc(q * sizeof(double**));
10     double*** Bblk = (double***)malloc(q * sizeof(double**));
11     double*** Cblk = (double***)malloc(q * sizeof(double**));
12     double*** curA = (double***)malloc(q * sizeof(double**));
13     double*** curB = (double***)malloc(q * sizeof(double**));
14     for (int i = 0; i < q; i++) {
15         Ablk[i] = (double**)malloc(q * sizeof(double*));
16         Bblk[i] = (double**)malloc(q * sizeof(double*));
17         Cblk[i] = (double**)malloc(q * sizeof(double*));
18         curA[i] = (double**)malloc(q * sizeof(double*));
19         curB[i] = (double**)malloc(q * sizeof(double*));
20     }
21     for (int i = 0; i < q; i++)
22         for (int j = 0; j < q; j++) {
23             Ablk[i][j] = (double*)0; /* placeholder pointers not used directly below */
24         }
25
26     /* Use simple 2D arrays of double** blocks instead (clearer) */
27     double**** Agrid = (double****)malloc(q * sizeof(double***));
28     double**** Bgrid = (double****)malloc(q * sizeof(double***));
29     double**** Cgrid = (double****)malloc(q * sizeof(double***));
30     for (int i = 0; i < q; i++) {
31         Agrid[i] = (double***)malloc(q * sizeof(double**));
32         Bgrid[i] = (double***)malloc(q * sizeof(double**));
33         Cgrid[i] = (double***)malloc(q * sizeof(double**));
34         for (int j = 0; j < q; j++) {
35             Agrid[i][j] = alloc_matrix(s);
36             Bgrid[i][j] = alloc_matrix(s);
37             Cgrid[i][j] = alloc_matrix(s);
38             for (int a = 0; a < s; a++)
39                 for (int b = 0; b < s; b++) {
40                     Agrid[i][j][a][b] = A[i*s+a][j*s+b];
```

```

41         Bgrid[i][j][a][b] = B[i*s+a][j*s+b];
42     }
43 }
44
45
46 /* Initial skew: skewA[i][j] = Agrid[i][(j+i)%q] ; skewB[i][j] = Bgrid[(i+j)%q][j] */
47 double**** curAg = (double****)malloc(q * sizeof(double****));
48 double**** curBg = (double****)malloc(q * sizeof(double****));
49 for (int i = 0; i < q; i++) {
50     curAg[i] = (double****)malloc(q * sizeof(double****));
51     curBg[i] = (double****)malloc(q * sizeof(double****));
52     for (int j = 0; j < q; j++) {
53         curAg[i][j] = Agrid[i][(j+i) % q];
54         curBg[i][j] = Bgrid[(i+j) % q][j];
55     }
56 }
57
58 for (int step = 0; step < q; step++) {
59     for (int i = 0; i < q; i++)
60         for (int j = 0; j < q; j++)
61             block_mult_accumulate(Cgrid[i][j], curAg[i][j], curBg[i][j], s);
62
63     /* circular shift A left (each proc takes from its right neighbor) */
64     double**** nextAg = (double****)malloc(q * sizeof(double****));
65     double**** nextBg = (double****)malloc(q * sizeof(double****));
66     for (int i = 0; i < q; i++) {
67         nextAg[i] = (double****)malloc(q * sizeof(double****));
68         nextBg[i] = (double****)malloc(q * sizeof(double****));
69         for (int j = 0; j < q; j++) {
70             nextAg[i][j] = curAg[i][(j+1) % q];
71             nextBg[i][j] = curBg[(i+1) % q][j];
72         }
73     }
74     for (int i = 0; i < q; i++) { free(curAg[i]); free(curBg[i]); }
75     free(curAg); free(curBg);
76     curAg = nextAg; curBg = nextBg;
77 }
78
79 /* Gather C */
80 for (int i = 0; i < q; i++)
81     for (int j = 0; j < q; j++)
82         for (int a = 0; a < s; a++)
83             for (int b = 0; b < s; b++)
84                 C[i*s+a][j*s+b] = Cgrid[i][j][a][b];
85
86 /* Cleanup */
87 for (int i = 0; i < q; i++) {
88     for (int j = 0; j < q; j++) {
89         free_matrix(Agrid[i][j], s);
90         free_matrix(Bgrid[i][j], s);
91         free_matrix(Cgrid[i][j], s);
92     }
93     free(Agrid[i]); free(Bgrid[i]); free(Cgrid[i]);
94     free(curAg[i]); free(curBg[i]);
95     free(Ablk[i]); free(Bblk[i]); free(Cblk[i]); free(curA[i]); free(curB[i]);
96 }
97 free(Agrid); free(Bgrid); free(Cgrid); free(curAg); free(curBg);
98 free(Ablk); free(Bblk); free(Cblk); free(curA); free(curB);
99 }
100
101 /* =====

```

Listing 10: CPU implementation

11 Fox's Algorithm (Broadcast-Multiply-Roll)

Pseudocode

Algorithm 11 Fox's Algorithm (Fox, Otto, Hey, 1987)

```
1: function FOXMATMUL( $A, B$ ) ▷ executed identically on each processor  $P_{i,j}$  in a  $\sqrt{p} \times \sqrt{p}$  grid
2:    $C_{i,j} \leftarrow 0$ 
3:   for  $step \leftarrow 0$  to  $\sqrt{p} - 1$  do
4:      $bcast\_col \leftarrow (i + step) \bmod \sqrt{p}$ 
5:     Processor  $P_{i, bcast\_col}$  broadcasts its  $A$ -block along row  $i$ 
6:      $C_{i,j} \leftarrow C_{i,j} + A_{broadcast} \times B_{local}$ 
7:     Circularly shift  $B_{local}$  one step up (within processor column)
8:   end for
9:   return  $C_{i,j}$ 
10: end function
```

Implementation note. Fox's Algorithm is inherently distributed-memory. The CPU implementation below simulates a $q \times q$ virtual processor grid, executing the broadcast-multiply-roll communication pattern sequentially.

CPU (C) Implementation — This Project

```
1  11. FOX'S ALGORITHM (broadcast-multiply-roll), simulated q x q grid
2  ===== */
3  void fox_matmul_simulated(double** A, double** B, double** C, int n, int q) {
4      if (n % q != 0) { fprintf(stderr, "[Fox] n must be divisible by q\n"); return; }
5      int s = n / q;
6      fill_zero(C, n);
7
8      double**** Agrid = (double****)malloc(q * sizeof(double***));
9      double**** Bgrid = (double****)malloc(q * sizeof(double***));
10     double**** Cgrid = (double****)malloc(q * sizeof(double***));
11     for (int i = 0; i < q; i++) {
12         Agrid[i] = (double****)malloc(q * sizeof(double**));
13         Bgrid[i] = (double****)malloc(q * sizeof(double**));
14         Cgrid[i] = (double****)malloc(q * sizeof(double**));
15         for (int j = 0; j < q; j++) {
16             Agrid[i][j] = alloc_matrix(s);
17             Bgrid[i][j] = alloc_matrix(s);
18             Cgrid[i][j] = alloc_matrix(s);
19             for (int a = 0; a < s; a++)
20                 for (int b = 0; b < s; b++) {
21                     Agrid[i][j][a][b] = A[i*s+a][j*s+b];
22                     Bgrid[i][j][a][b] = B[i*s+a][j*s+b];
23                 }
24         }
25     }
26
27     double**** curBg = (double****)malloc(q * sizeof(double***));
28     for (int i = 0; i < q; i++) {
29         curBg[i] = (double****)malloc(q * sizeof(double**));
30         for (int j = 0; j < q; j++) curBg[i][j] = Bgrid[i][j];
31     }
32
33     for (int step = 0; step < q; step++) {
34         for (int i = 0; i < q; i++) {
35             int bcast_col = (i + step) % q;
36             double** Abroadcast = Agrid[i][bcast_col]; /* row i broadcast source */
37             for (int j = 0; j < q; j++)
38                 block_mult_accumulate(Cgrid[i][j], Abroadcast, curBg[i][j], s);
39         }
40         /* shift B up: each row takes B from the row below it (wraparound) */
41         double**** nextBg = (double****)malloc(q * sizeof(double***));
42         for (int i = 0; i < q; i++) {
```

```

43     nextBg[i] = (double***)malloc(q * sizeof(double**));
44     for (int j = 0; j < q; j++) nextBg[i][j] = curBg[(i+1) % q][j];
45 }
46 for (int i = 0; i < q; i++) free(curBg[i]);
47 free(curBg);
48 curBg = nextBg;
49 }
50
51 for (int i = 0; i < q; i++)
52     for (int j = 0; j < q; j++)
53         for (int a = 0; a < s; a++)
54             for (int b = 0; b < s; b++)
55                 C[i*s+a][j*s+b] = Cgrid[i][j][a][b];
56
57 for (int i = 0; i < q; i++) {
58     for (int j = 0; j < q; j++) {
59         free_matrix(Agrid[i][j], s);
60         free_matrix(Bgrid[i][j], s);
61         free_matrix(Cgrid[i][j], s);
62     }
63     free(Agrid[i]); free(Bgrid[i]); free(Cgrid[i]); free(curBg[i]);
64 }
65 free(Agrid); free(Bgrid); free(Cgrid); free(curBg);
66 }
67
68 /* =====

```

Listing 11: CPU implementation

12 SUMMA (Scalable Universal Matrix Multiplication Algorithm)

Pseudocode

Algorithm 12 SUMMA (van de Geijn & Watts, 1997)

```
1: function SUMMA( $A, B$ )                                ▷ executed identically on each processor  $P_{i,j}$  in a  $p_r \times p_c$  grid
2:    $C_{i,j} \leftarrow 0$ 
3:   for  $k \leftarrow 0$  to  $n - 1$  do                        ▷ or in blocks of width  $b$  for blocked SUMMA
4:     The processor column owning column  $k$  of  $A$  broadcasts that column ( $A_{:,k}$ ) across its processor row
5:     The processor row owning row  $k$  of  $B$  broadcasts that row ( $B_{k,:}$ ) across its processor column
6:      $C_{i,j} \leftarrow C_{i,j} + A_{i,k} \cdot B_{k,j}$         ▷ rank-1 update
7:   end for
8:   return  $C_{i,j}$ 
9: end function
```

Implementation note. SUMMA is inherently distributed-memory. The CPU implementation below simulates a $q \times q$ virtual processor grid using SUMMA's broadcast-then-rank-1-update pattern, structurally distinct from Cannon's/Fox's shift-based approach. The genuine author-provided PLAPACK source is also included above for reference.

Author-Provided Original Source Code (SUMMA)

This is the actual production C source from Robert van de Geijn's PLAPACK library (GNU GPL), directly implementing the SUMMA algorithm he co-authored. Verified against the live file at https://www.cs.utexas.edu/~plapack/Code/PLAPACK20/BLAS3/PLA_Gemm.c.

```
1  /*****
2  Parallel Linear Algebra Package Release R2.0.2 -- 6 Feb 2000
3  Copyright (c) 1997,1998,1999,2000 Robert van de Geijn and
4  The University of Texas at Austin.  (GNU General Public License)
5
6  Written under the direction of: Robert van de Geijn,
7  Department of Computer Sciences, University of Texas at Austin
8
9  SOURCE: https://www.cs.utexas.edu/~plapack/Code/PLAPACK20/BLAS3/PLA_Gemm.c
10 *****/
11
12 #include "PLA.h"
13
14 int PLA_Gemm( int transa, int transb,
15               PLA_Obj alpha, PLA_Obj A, PLA_Obj B,
16               PLA_Obj beta, PLA_Obj C )
17 /*
18  Purpose : Parallel matrix multiplication
19  C <- alpha * A * B + beta * C    (plus transposed variants)
20
21  NOTE: For details on how to implement matrix-matrix multiplication, see
22  R. van de Geijn, Using PLAPACK, The MIT Press, 1997.
23  R. van de Geijn and J. Watts,
24  "SUMMA: Scalable Universal Matrix Multiplication Algorithm,"
25  Concurrency: Practice and Experience, Vol 9(4), pp. 255-274 (1997).
26 */
27 {
28   int value = PLA_SUCCESS, owner_row, owner_col;
29   int length_A, width_A, length_B, width_B, length_C, width_C, nb_alg;
30   PLA_Obj alpha_cpy = NULL, beta_cpy = NULL;
31   PLA_Template templ = NULL;
32
33   if ( PLA_ERROR_CHECKING )
34     value = PLA_Gemm_enter( transa, transb, alpha, A, B, beta, C );
35
36   if ( !value ){
37     /* If necessary, duplicate alpha and beta to all nodes */
38     PLA_Obj_owner_row( alpha, &owner_row );
```

```

39 PLA_Obj_owner_col( alpha, &owner_col );
40 if ( owner_row != PLA_ALL_ROWS || owner_col != PLA_ALL_COLS ){
41     PLA_Mscalar_create_conf_to( alpha, PLA_ALL_ROWS, PLA_ALL_COLS, &alpha_cpy );
42     PLA_Copy( beta, beta_cpy );
43 }
44 PLA_Obj_owner_row( beta, &owner_row );
45 PLA_Obj_owner_col( beta, &owner_col );
46 if ( owner_row != PLA_ALL_ROWS || owner_col != PLA_ALL_COLS ){
47     PLA_Mscalar_create_conf_to( beta, PLA_ALL_ROWS, PLA_ALL_COLS, &beta_cpy );
48     PLA_Copy( beta, beta_cpy );
49 }
50
51 PLA_Obj_template( A, &templ );
52 PLA_Obj_global_length( A, &length_A ); PLA_Obj_global_width( A, &width_A );
53 PLA_Obj_global_length( B, &length_B ); PLA_Obj_global_width( B, &width_B );
54 PLA_Obj_global_length( C, &length_C ); PLA_Obj_global_width( C, &width_C );
55
56 /* SUMMA's core idea: whichever matrix (A, B, or C) holds the most data
57    is left in place, and the other two are the ones communicated/broadcast */
58 if ( length_A*width_A > length_B*width_B && length_A*width_A > length_C*width_C ){
59     PLA_Environ_nb_alg( PLA_OP_MAT_PAN, templ, &nb_alg );
60     PLA_Gemm_A( nb_alg, transa, transb,
61                 (alpha_cpy==NULL?alpha:alpha_cpy), A, B,
62                 (beta_cpy==NULL?beta:beta_cpy), C );
63 }
64 else if ( length_B*width_B > length_A*width_A && length_B*width_B > length_C*width_C ){
65     PLA_Environ_nb_alg( PLA_OP_PAN_MAT, templ, &nb_alg );
66     PLA_Gemm_B( nb_alg, transa, transb,
67                 (alpha_cpy==NULL?alpha:alpha_cpy), A, B,
68                 (beta_cpy==NULL?beta:beta_cpy), C );
69 }
70 else {
71     PLA_Environ_nb_alg( PLA_OP_PAN_PAN, templ, &nb_alg );
72     PLA_Gemm_C( nb_alg, transa, transb,
73                 (alpha_cpy==NULL?alpha:alpha_cpy), A, B,
74                 (beta_cpy==NULL?beta:beta_cpy), C );
75 }
76 PLA_Obj_free( &alpha_cpy );
77 PLA_Obj_free( &beta_cpy );
78 }
79 if ( PLA_ERROR_CHECKING )
80     value = PLA_Gemm_exit( transa, transb, alpha, A, B, beta, C );
81 return value;
82 }

```

Listing 12: Author-original source code

CPU (C) Implementation — This Project

```

1 12. SUMMA (broadcast column-of-A-blocks / row-of-B-blocks, rank-b update)
2 Simulated over a q x q grid (pr = pc = q here for simplicity).
3 ===== */
4 void summa_matmul_simulated(double** A, double** B, double** C, int n, int pr, int pc) {
5     if (pr != pc) { fprintf(stderr, "[SUMMA] demo requires pr == pc\n"); return; }
6     int q = pr;
7     if (n % q != 0) { fprintf(stderr, "[SUMMA] n must be divisible by grid size\n"); return; }
8     int s = n / q;
9     fill_zero(C, n);
10
11     double**** Agrid = (double****)malloc(q * sizeof(double****));
12     double**** Bgrid = (double****)malloc(q * sizeof(double****));
13     double**** Cgrid = (double****)malloc(q * sizeof(double****));
14     for (int i = 0; i < q; i++) {
15         Agrid[i] = (double****)malloc(q * sizeof(double****));
16         Bgrid[i] = (double****)malloc(q * sizeof(double****));
17         Cgrid[i] = (double****)malloc(q * sizeof(double****));
18         for (int j = 0; j < q; j++) {
19             Agrid[i][j] = alloc_matrix(s);

```

```

20     Bgrid[i][j] = alloc_matrix(s);
21     Cgrid[i][j] = alloc_matrix(s);
22     for (int a = 0; a < s; a++)
23         for (int b = 0; b < s; b++) {
24             Agrid[i][j][a][b] = A[i*s+a][j*s+b];
25             Bgrid[i][j][a][b] = B[i*s+a][j*s+b];
26         }
27     }
28 }
29
30 /* At each panel step kp, the processor column owning A-block-column kp
31    "broadcasts" A[i][kp] across row i; the processor row owning
32    B-block-row kp "broadcasts" B[kp][j] across column j. In this
33    sequential simulation, broadcasting simply means every processor
34    reads that shared block directly (no physical copy needed on a
35    single machine) -- this is the key structural difference from
36    Cannon/Fox, which instead physically SHIFT blocks between steps. */
37 for (int kp = 0; kp < q; kp++) {
38     for (int i = 0; i < q; i++)
39         for (int j = 0; j < q; j++)
40             block_mult_accumulate(Cgrid[i][j], Agrid[i][kp], Bgrid[kp][j], s);
41 }
42
43 for (int i = 0; i < q; i++)
44     for (int j = 0; j < q; j++)
45         for (int a = 0; a < s; a++)
46             for (int b = 0; b < s; b++)
47                 C[i*s+a][j*s+b] = Cgrid[i][j][a][b];
48
49 for (int i = 0; i < q; i++) {
50     for (int j = 0; j < q; j++) {
51         free_matrix(Agrid[i][j], s);
52         free_matrix(Bgrid[i][j], s);
53         free_matrix(Cgrid[i][j], s);
54     }
55     free(Agrid[i]); free(Bgrid[i]); free(Cgrid[i]);
56 }
57 free(Agrid); free(Bgrid); free(Cgrid);
58 }
59
60 /* =====

```

Listing 13: CPU implementation

13 Communication-Avoiding Matrix Multiplication (CA-MM / 2.5D)

Pseudocode

Algorithm 13 Communication-Avoiding (2.5D) Matrix Multiplication (Ballard, Demmel, Holtz, Schwartz, 2011)

- 1: **function** CA25DMATMUL(A, B, c) $\triangleright c$ = replication factor; p processors arranged as $\sqrt{p/c} \times \sqrt{p/c} \times c$
- 2: Partition the $\sqrt{p/c} \times \sqrt{p/c}$ processor grid into c layers
- 3: Distribute A, B blocks across the base 2-D grid; replicate across the c layers
- 4: Each of the c layers independently performs $1/c$ of the total SUMMA-style (or Cannon-style) computation, using only its local $1/c$ share of the k -dimension
- 5: Perform a reduction (sum) across the c layers to combine partial results into the final block of C
- 6: **return** C $\triangleright$ achieves the $\Omega(n^2/\sqrt{M})$ communication lower bound
- 7: **end function**

Implementation note. Communication-Avoiding (2.5D) Matrix Multiplication is inherently distributed-memory. The CPU implementation below simulates the c -layer replication and final reduction step that characterizes the 2.5D approach.

CPU (C) Implementation — This Project

```
1  13. COMMUNICATION-AVOIDING MATRIX MULTIPLICATION (2.5D)
2  The shared (k) dimension is split into c layers; each layer computes
3  an independent partial product using only its 1/c share of k, then
4  all c partial results are summed (the "reduction" step) to form C.
5  This models the 2.5D algorithm's core idea: trade extra memory
6  (c-way replication) for reduced communication volume.
7  ===== */
8  void ca_mm_2_5d_simulated(double** A, double** B, double** C, int n, int q, int c) {
9      (void)q; /* base grid dimension retained for interface symmetry/documentation */
10     if (n % c != 0) { fprintf(stderr, "[CA-MM 2.5D] n must be divisible by c\n"); return; }
11     int kchunk = n / c;
12     fill_zero(C, n);
13
14     double** partial = alloc_matrix(n);
15     for (int layer = 0; layer < c; layer++) {
16         fill_zero(partial, n);
17         int k0 = layer * kchunk, k1 = k0 + kchunk;
18         for (int i = 0; i < n; i++) {
19             for (int j = 0; j < n; j++) {
20                 double sum = 0.0;
21                 for (int k = k0; k < k1; k++) sum += A[i][k] * B[k][j];
22                 partial[i][j] = sum;
23             }
24         }
25         /* Reduction across layers: accumulate into C */
26         for (int i = 0; i < n; i++)
27             for (int j = 0; j < n; j++)
28                 C[i][j] += partial[i][j];
29     }
30     free_matrix(partial, n);
31 }
```

Listing 14: CPU implementation

14 Parallel GEMM (General Category)

Pseudocode

Algorithm 14 Parallel GEMM — Generic Driver (dispatches to an underlying algorithm)

```
1: function PARALLELGEMM( $A, B$ , algorithm)
2:   if algorithm = Cannon then
3:     return CANNONMATMUL( $A, B$ )
4:   else if algorithm = Fox then
5:     return FOXMATMUL( $A, B$ )
6:   else if algorithm = SUMMA then
7:     return SUMMA( $A, B$ )
8:   else if algorithm = CA-MM (2.5D) then
9:     return CA25DMATMUL( $A, B, c$ )
10:  end if
11: end function
```

Implementation note. Parallel GEMM is an umbrella category, not a single algorithm. The implementation below is a dispatcher over the four concrete distributed-memory algorithms implemented above.

CPU (C) Implementation — This Project

```
1  14. PARALLEL GEMM -- general umbrella category, not a single algorithm.
2  This dispatcher demonstrates that "Parallel GEMM" simply means picking
3  one of the concrete distributed-memory algorithms above (Cannon, Fox,
4  SUMMA, or CA-MM/2.5D) as the underlying communication scheme.
5  ===== */
6  void parallel_gemm(double** A, double** B, double** C, int n, ParallelGemmAlgo algo) {
7      int q = 2; /* demo grid dimension; n must be divisible by q (and by q*c for CA-MM) */
8      switch (algo) {
9          case PGEMM_CANNON:
10             cannon_matmul_simulated(A, B, C, n, q);
11             break;
12          case PGEMM_FOX:
13             fox_matmul_simulated(A, B, C, n, q);
14             break;
15          case PGEMM_SUMMA:
16             summa_matmul_simulated(A, B, C, n, q, q);
17             break;
18          case PGEMM_CA25D:
19             ca_mm_2_5d_simulated(A, B, C, n, q, q); /* c = q for this demo */
20             break;
21          default:
22             fprintf(stderr, "[Parallel GEMM] Unknown algorithm selector\n");
23      }
24  }
25
26  /* =====
```

Listing 15: CPU implementation

15 cuBLAS GEMM (Optimized Library Implementation – Not a Novel Algorithm)

Pseudocode

Algorithm 15 cuBLAS-style GEMM — High-Level Structure (library-internal, not a new algorithm)

- 1: **function** cuBLAS_GEMM(A, B, C, α, β)
 - 2: Select a pre-tuned kernel variant based on GPU architecture, matrix shape, and data type (FP32/FP16/BF16/TF32/INT8, via autotuning/heuristics)
 - 3: Load tiles of A and B into shared memory using warp-level and register-blocked tiling (conceptually an optimized version of TILED MATMUL KERNEL)
 - 4: On architectures with Tensor Cores, issue `mma/wgmma` warp-matrix-multiply-accumulate instructions instead of scalar FMA
 - 5: Accumulate partial tile products into registers/shared memory
 - 6: Apply the epilogue: $C \leftarrow \alpha \cdot (A \times B) + \beta \cdot C$
 - 7: **return** C
 - 8: **end function**
-

Implementation note. cuBLAS is closed-source and GPU-only; it cannot run on a CPU and is not reimplemented here. The implementation below is an aggressively blocked, cache-friendly CPU GEMM used only as a *best-practical-effort performance analog*, not a reimplementation of cuBLAS. The closest official NVIDIA open-source code (CUTLASS) is included above.

Closest Official NVIDIA Source Code (CUTLASS, cuBLAS-Adjacent)

cuBLAS itself is closed-source and cannot be reproduced. NVIDIA’s own engineering team states that CUTLASS “incorporates strategies for hierarchical decomposition and data movement similar to those used to implement cuBLAS and cuDNN.” This is NVIDIA’s official open-source (BSD-3-Clause) GEMM reference, verified against https://github.com/NVIDIA/cutlass/blob/main/examples/00_basic_gemm/basic_gemm.cu.

```
1  /*****
2  * Copyright (c) 2017 - 2026 NVIDIA CORPORATION & AFFILIATES. All rights reserved.
3  * SPDX-License-Identifier: BSD-3-Clause
4  *
5  * SOURCE: https://github.com/NVIDIA/cutlass/blob/main/examples/00_basic_gemm/basic_gemm.cu
6  *
7  * NOTE: cuBLAS itself is closed-source. NVIDIA officially states that
8  * CUTLASS "incorporates strategies for hierarchical decomposition and
9  * data movement similar to those used to implement cuBLAS and cuDNN."
10 * This is the closest official, open-source NVIDIA GEMM code available.
11 *****/
12
13 #include "cutlass/gemm/device/gemm.h"
14
15 /// Define a CUTLASS GEMM template and launch a GEMM kernel.
16 cudaError_t CutlassSgemmNN(
17     int M,
18     int N,
19     int K,
20     float alpha,
21     float const *A,
22     int lda,
23     float const *B,
24     int ldb,
25     float beta,
26     float *C,
27     int ldc) {
28
29     // Define type definition for single-precision CUTLASS GEMM with column-major
30     // input matrices and 128x128x8 threadblock tile size (chosen by default).
31     using ColumnMajor = cutlass::layout::ColumnMajor;
```

```

32
33 using CutlassGemm = cutlass::gemm::device::Gemm<float,           // Data-type of A matrix
34                                     ColumnMajor, // Layout of A matrix
35                                     float,       // Data-type of B matrix
36                                     ColumnMajor, // Layout of B matrix
37                                     float,       // Data-type of C matrix
38                                     ColumnMajor>; // Layout of C matrix
39
40 CutlassGemm gemm_operator;
41
42 // Construct the CUTLASS GEMM arguments object.
43 CutlassGemm::Arguments args({M, N, K}, // Gemm Problem dimensions
44                             {A, lda},  // Tensor-ref for source matrix A
45                             {B, ldb},  // Tensor-ref for source matrix B
46                             {C, ldc},  // Tensor-ref for source matrix C
47                             {C, ldc},  // Tensor-ref for destination matrix D
48                             {alpha, beta}); // Scalars used in the Epilogue
49
50 // Launch the CUTLASS GEMM kernel.
51 cutlass::Status status = gemm_operator(args);
52
53 if (status != cutlass::Status::kSuccess) {
54     return cudaErrorUnknown;
55 }
56 return cudaSuccess;
57 }
58
59 /// Naive reference GEMM computation (included in the same official file,
60 /// used by NVIDIA to verify the CUTLASS kernel's correctness).
61 __global__ void ReferenceGemm_kernel(
62     int M,
63     int N,
64     int K,
65     float alpha,
66     float const *A,
67     int lda,
68     float const *B,
69     int ldb,
70     float beta,
71     float *C,
72     int ldc) {
73
74     int i = threadIdx.x + blockIdx.x * blockDim.x;
75     int j = threadIdx.y + blockIdx.y * blockDim.y;
76
77     if (i < M && j < N) {
78         float accumulator = 0;
79
80         for (int k = 0; k < K; ++k) {
81             accumulator += A[i + k * lda] * B[k + j * ldb];
82         }
83
84         C[i + j * ldc] = alpha * accumulator + beta * C[i + j * ldc];
85     }
86 }

```

Listing 16: Author-original source code

CPU (C) Implementation — This Project

```

1  15. cuBLAS GEMM -- GPU-only, closed-source. It CANNOT run on a CPU and
2  is not reimplemented here (that would misrepresent it). What follows
3  is an aggressively blocked, cache- and loop-order-optimized CPU GEMM
4  used purely as a "best practical CPU analog" performance reference
5  point -- NOT a reimplementation of cuBLAS, and NOT claimed to be
6  equivalent to it. The real GPU-side reference implementation
7  NVIDIA officially provides is CUTLASS (see README / earlier chat).
8  ===== */

```

```

9 void cublas_style_cpu_reference_gemm(double** A, double** B, double** C, int n) {
10     fprintf(stderr,
11         "[NOTE] cuBLAS itself is closed-source and GPU-only; it cannot run "
12         "on a CPU and is not reimplemented here. This function is only a "
13         "best-practical-effort CPU analog (blocked + cache-friendly loop "
14         "order), used as a performance reference point.\n");
15
16     const int BS = 32; /* block size tuned for typical L1/L2 cache */
17     fill_zero(C, n);
18     for (int ii = 0; ii < n; ii += BS) {
19         int imax = (ii + BS < n) ? ii + BS : n;
20         for (int kk = 0; kk < n; kk += BS) {
21             int kmax = (kk + BS < n) ? kk + BS : n;
22             for (int jj = 0; jj < n; jj += BS) {
23                 int jmax = (jj + BS < n) ? jj + BS : n;
24                 /* i-k-j loop order: inner j-loop is stride-1 on both
25                  * B[k][j] and C[i][j], maximizing cache-line reuse --
26                  * this is the single biggest lever available on CPU
27                  * before resorting to explicit SIMD intrinsics. */
28                 for (int i = ii; i < imax; i++) {
29                     for (int k = kk; k < kmax; k++) {
30                         double a_ik = A[i][k];
31                         for (int j = jj; j < jmax; j++) {
32                             C[i][j] += a_ik * B[k][j];
33                         }
34                     }
35                 }
36             }
37         }
38     }
39 }

```

Listing 17: CPU implementation